

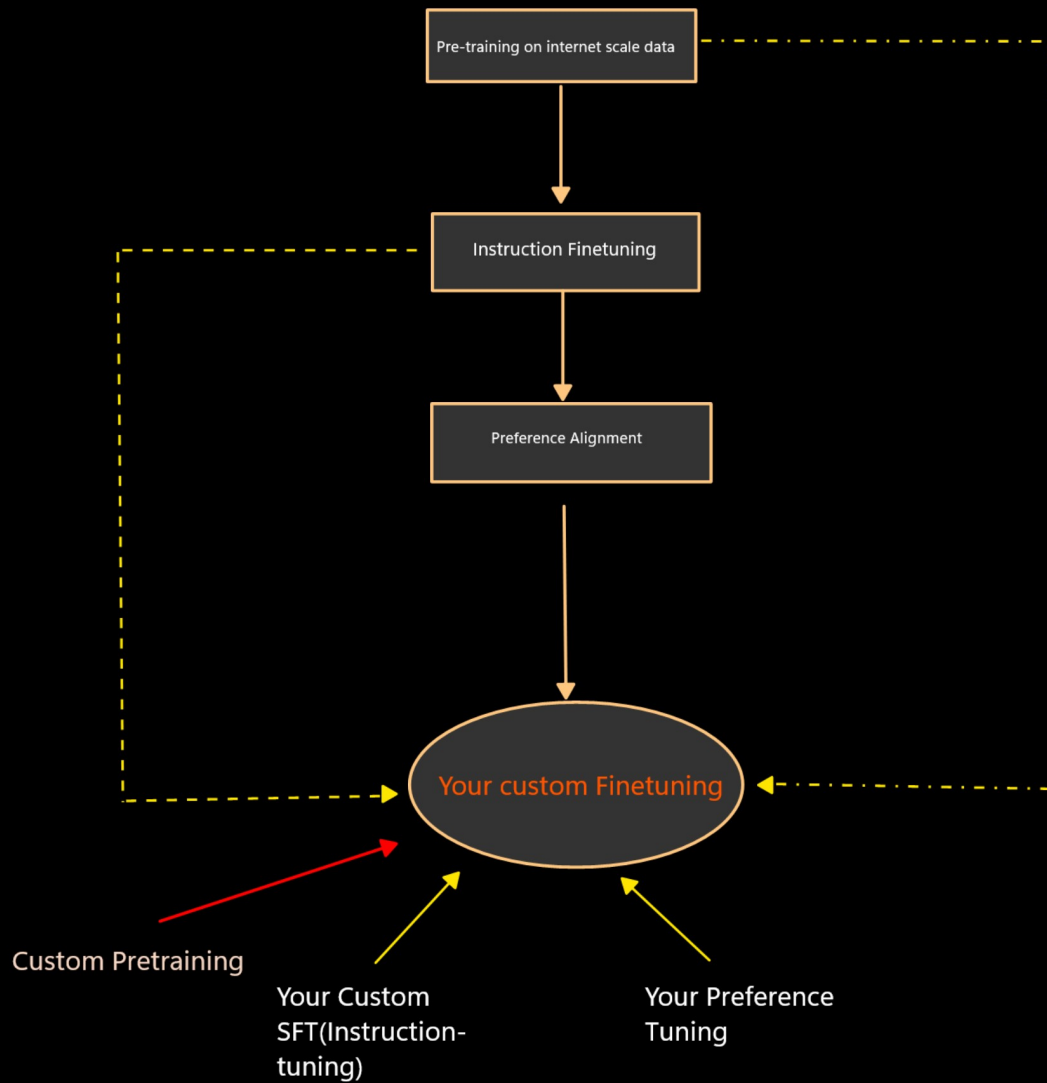
Understand LLM training
Transfer learning and model fine-tuning
Why fine-tuning was difficult in LSTM models
LSTM vs Transformer
Fine-tuning BERT
LLM knowledge distillation
LLM quantization
LLM fine-tuning with non-instructional data (raw PDF, TXT, documents)
Instruction fine-tuning
Preference alignment
Fine-tuning using tools: Hugging Face, LLaMA Factory, Unsloth, Axolotl
GPT fine-tuning
Gemini fine-tuning
SLM fine-tuning
Multimodal fine-tuning
Embedding fine-tuning

LORA
RLHF(PPO)
GRPO
DPO
ORPO

1. Training stages of LLMs and where this LoRA exist?
2. Full Parameter Fine-Tuning vs PEFT (Parameter Efficient Fine-Tuning) and methods to achieve PEFT
3. What are Weights in Neural Networks and Transformer Architecture?
4. Matrix and Rank in Matrix
5. What is LoRA or LoRA Adapter and benefits of using LoRA
6. What is Optimizer (Weight Update Concept)?
7. Practical of LORA

✓
LoRA | RLHF ✓

Understand LLM training



meta-llama/Llama-3.2-1B ← BASE
meta-llama/Llama-3.2-1B-Instruct ← Instr
alignment-handbook/zephyr-7b-dpo-full ← Preference

Own custom model
WRT all 3 stage ⇒ Y/N

Full-parameter Fine-tuning

Updates all the model weights

The weight matrices are very large

7B model has 7 billion weights, 13B has 13 billion, and so on

All weights get updated repeatedly for multiple "epochs"

Storing and updating weights requires a lot of memory
which limits fine-tuning to large GPUs or GPU clusters

Expensive Training

Parameter-Efficient Fine-tuning (PEFT)

Does not update all the model weights

Original weight matrices are frozen

Only a small number of additional weights(parameters) are trained

Way of achieve PEFT:

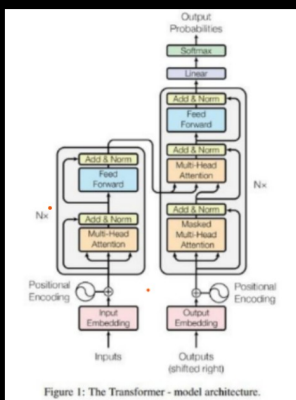
1. LoRA (Low-Rank Adaptation) → QLoRA (Quantized Low-Rank Adaptation)
2. DoRA (Weight-Decomposed Low-Rank Adaptation)
3. Prefix Tuning (adds learnable prefix vectors)
4. Prompt Tuning (learns input embeddings)
5. BitFit (trains only bias parameters)
6. IA³ (scales attention mechanisms)

Example: LoRA trains small low-rank matrices instead of full weights Training updates only these few parameters repeatedly for multiple epochs

LoRA = Less memory + Faster training + Lower cost + Easy customization

enabling fine-tuning on single GPUs or even consumer hardware

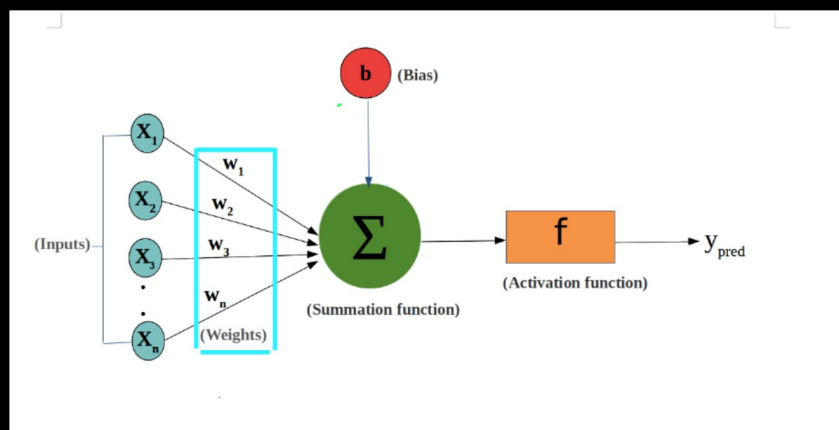
Performance almost same



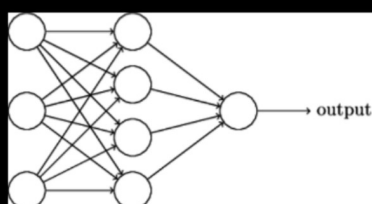
What is weights ?

Weight = importance score

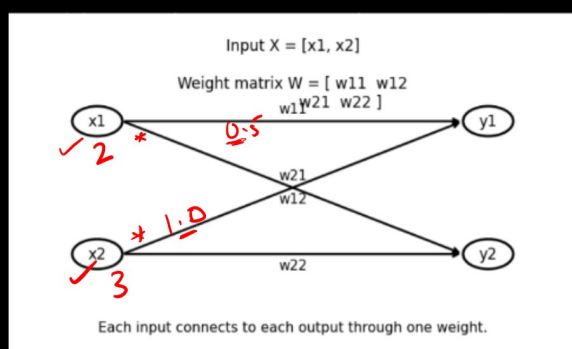
Neural Net: "Which feature is important?"



$$W = \begin{bmatrix} w_1 \\ w_2 \\ w_3 \\ \vdots \\ w_n \end{bmatrix}$$



$$W = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \\ w_{41} & w_{42} & w_{43} \end{bmatrix}$$



$$W = \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{bmatrix}$$

Example:

Input:

$x_1 = 2, x_2 = 3$

Weights:

$w_1 = 0.5, w_2 = 1.0$

What happens during training?

Initially weights are random

During training:

- Loss is calculated
- *Weights are updated using gradients*

Goal:

Find the best weights

init → randomly
training → adjust → loss!!!

SF

Adjust

NN / transformer

FP

weight & bias

BP

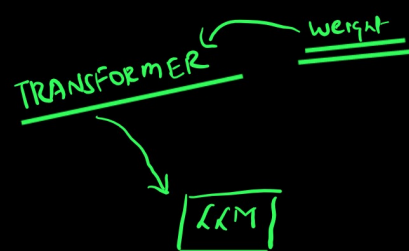
optimizer

Loss

Imp score

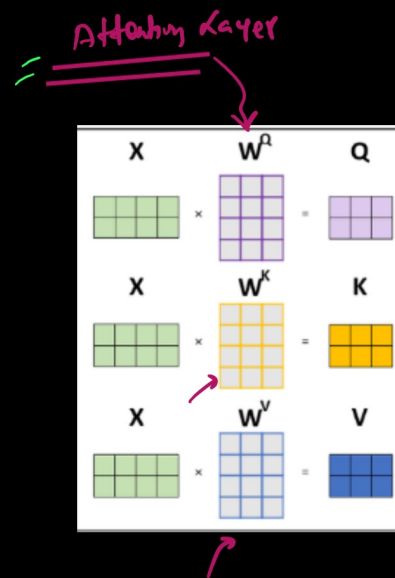
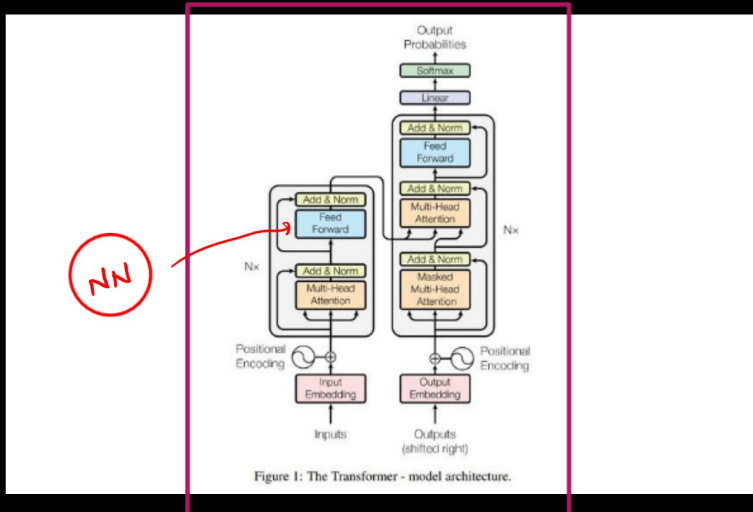
$$\text{Taste} = \text{Patty} \times 0.9 + \text{Cheese} \times 0.7 + \text{Sauce} \times 0.5 + \text{Bun} \times 0.2$$

TASTE o/p



Weights in Transformer Architecture

Transformer: "Which word is important for another word?"



Linear Layers

- WQ (Query)
- WK (Key)
- WV (Value)

These are all weights matrices

Attention Weights (MOST IMPORTANT)

Model decides:

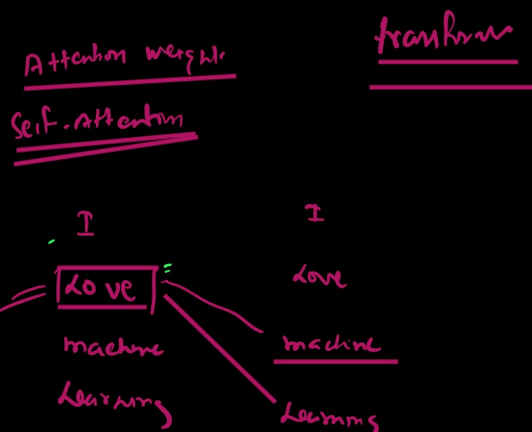
Which word is important for another word

"I love machine learning"

"learning" focuses on which word?

- love? ✓
- machine? ✓✓

This importance = attention weights



NN & transformer

weights

{ }

Deep Learning = A Matrix Game

Output = Input × Weights + Bias

AI → matrix
DL

Input → vector

Weights(Model parameters) → matrix

Output → transformed vector

Images → matrices (pixels)

Videos → 3D matrices

Text Embeddings → Words → vectors

Attention (Transformers)

Q, K, V → matrices

Attention = matrix multiplication

Matrix and Rank in Matrix

Differences between scalar, vector and matrix

Scalar 5

Vector [1,2,3]

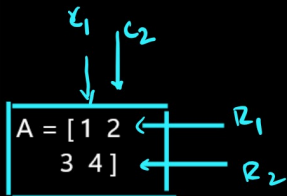
Matrix $\begin{bmatrix} 1,2 \\ 3,4 \end{bmatrix}$

Grid of value

Matrix = a table of numbers (grid)

Matrix = vectors ka collection

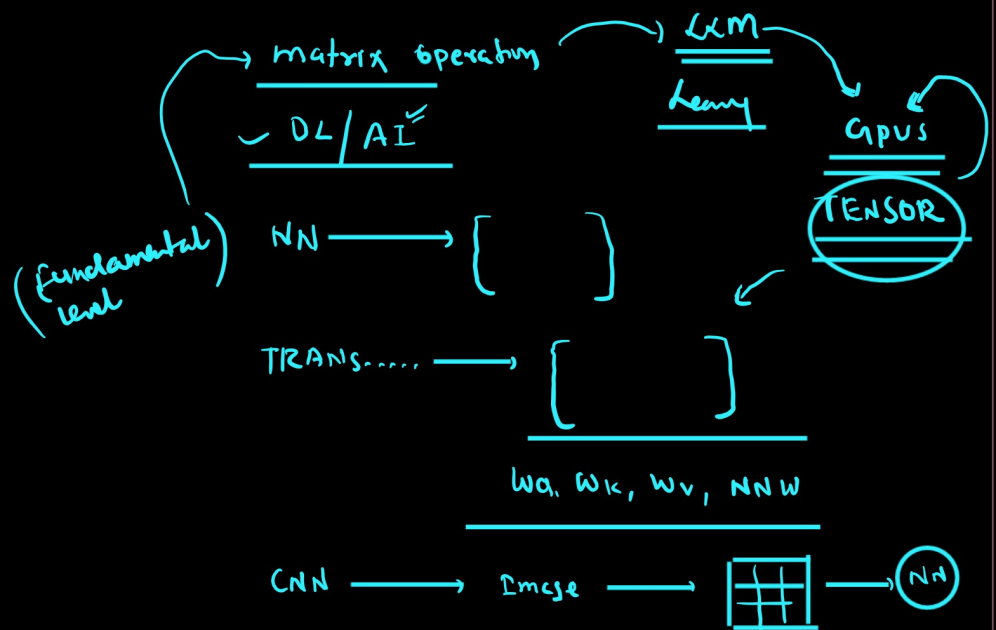
Vector = special case of matrix (1 row or 1 column)



This is called a 2x2 matrix.

✓ 2 rows

✓ 2 columns



More Examples

1. 3×2 Matrix

$$B = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$$

This is called a 3×2 matrix.

3 rows

2 columns

2. 2×3 Matrix

$$C = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

This is called a 2×3 matrix.

2 rows

3 columns

3. $m \times n$ Matrix (General Form)

$$D = \begin{bmatrix} a_{11} & a_{12} & a_{13} & \dots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \dots & a_{2n} \\ \dots & & & & \\ a_{m1} & a_{m2} & a_{m3} & \dots & a_{mn} \end{bmatrix}$$

This is called an $m \times n$ matrix.

m rows

n columns

Row Matrix

$$R = [1 \ 2 \ 3 \ 4]$$

This is called a 1×4 matrix.

1 row

4 columns

Only one row $\rightarrow$ so it is a row matrix

Column Matrix

$$C = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix}$$

This is called a 4×1 matrix.

4 rows

1 column

Only one column $\rightarrow$ so it is a column matrix



In simple language: Number of linearly independent rows or columns

How many linearly independent row or column are there?

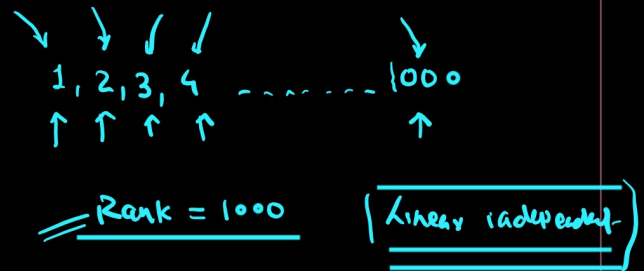
Only 2–3 types of people, just scaled differently

If both go in same direction $\rightarrow$ rank = 1

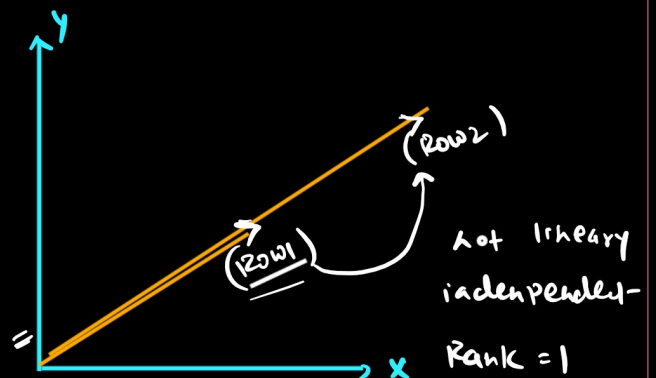
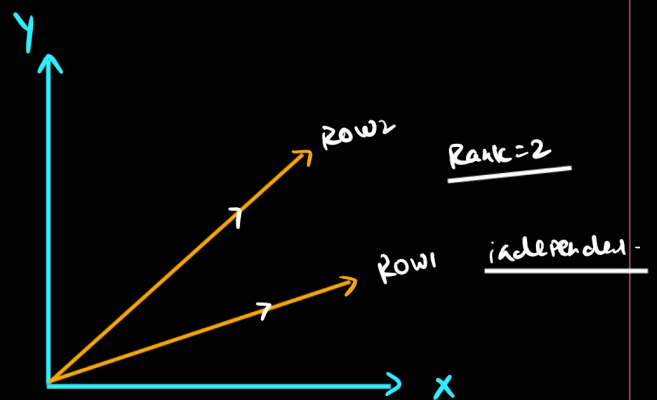
1000×1000 → max rank 1000

Maximum rank = $\min(m,n)$

- Can one row be formed from another row?
- If yes → information is repeating, Rank is low



People → 3

Rank $\rightarrow 3$ 

$\{ \}$
 kuxiwo $\longrightarrow$ Full rank = 100

Example 1: Full Rank Matrix

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

Let's check:

Is second row a multiple of first row?

First row: (1,2)

If we multiply by 3:

(3,6)

But second row is (3,4)

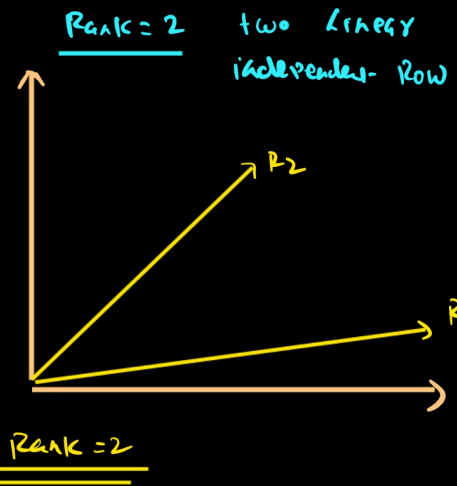
It does not match

Rows are independent

Rank = 2

Full rank

$$\begin{aligned} & \xrightarrow{(R_1)} 2 \begin{bmatrix} 1, 2 \end{bmatrix} \rightarrow \begin{bmatrix} 2, 4 \end{bmatrix} \\ & 3 \begin{bmatrix} 1, 2 \end{bmatrix} \rightarrow \begin{bmatrix} 3, 6 \end{bmatrix} \\ & \neq \begin{bmatrix} 3, 4 \end{bmatrix} R_2 \end{aligned}$$



Example 2: Low Rank Matrix

$$B = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$$

Second row = 2 × first row

(1,2) × 2 = (2,4)

Second row is not independent

Rank = 1

This is a low rank matrix

Understand from column view
Matrix:

$$B = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$$

Columns:

Column1 = (1,2)

Column2 = (2,4)

Column2 = 2 × Column1

Columns are also dependent

So rank = 1

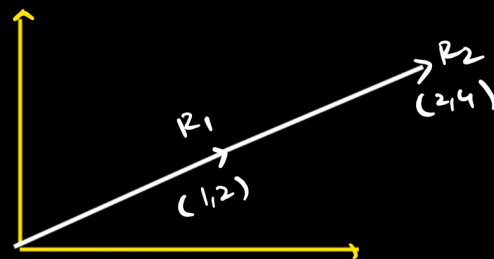
$$R_1 = \begin{bmatrix} 1, 2 \end{bmatrix}$$

$$R_2 = \begin{bmatrix} 2, 4 \end{bmatrix} \rightarrow \underline{2 \times R_1} \quad 2 \begin{bmatrix} 1, 2 \end{bmatrix}$$

$$C_1 = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

$$C_2 = \begin{bmatrix} 2 \\ 4 \end{bmatrix} \rightarrow \underline{2 \times C_1} \rightarrow 2 \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

Rank = 1 ← one linearly independent Row



Example 3: 3x3 example

$$C = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 3 & 6 & 9 \end{bmatrix} \quad \text{3 Row, 3 Col}$$

Notice:

$$\text{Row2} = 2 \times \text{Row1} \quad 2 \begin{bmatrix} 1 & 2 & 3 \end{bmatrix}$$

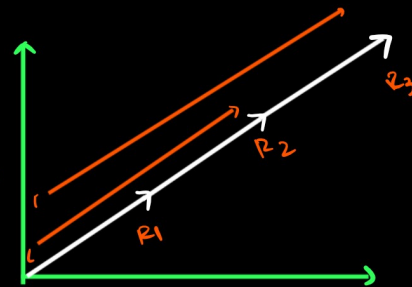
$$\text{Row3} = 3 \times \text{Row1} \quad 3 \begin{bmatrix} 1 & 2 & 3 \end{bmatrix}$$

All rows are in the same direction

Rank = 1

ONE Rank = 1

Linear independent -
Row $\rightarrow$ 1



Example 4: 3x3 example

Matrix:

D =

$$\begin{bmatrix} 1 & 2 & 3 \end{bmatrix} \leftarrow R_1$$

$$\begin{bmatrix} 4 & 5 & 6 \end{bmatrix} \leftarrow R_2$$

$$\begin{bmatrix} 7 & 8 & 9 \end{bmatrix} \leftarrow R_3$$

Reason:

Rows are not fully independent. There is a linear relation:

$$\text{Row3} = 2 \times \text{Row2} - \text{Row1}$$

because

$$2(4,5,6) - (1,2,3) = (8,10,12) - (1,2,3) = (7,8,9)$$

This means the 3rd row is dependent.

But:

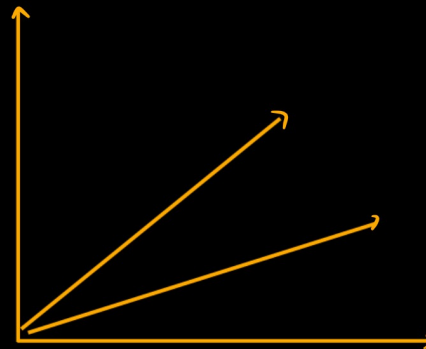
Row1 and Row2 are independent

2 linearly independent row

Therefore total independent rows = 2

Final Answer: Rank = 2

$$\begin{aligned} [7 \ 8 \ 9] &= 2[4 \ 5 \ 6] - [1 \ 2 \ 3] \\ &= [7 \ 8 \ 9] \end{aligned}$$



Rank R or C $\rightarrow$ Same

Now the question is what to see to check the rank of the matrix? Row or column?
you can check both — the result will be the same

Concept: Rank = number of linearly independent rows = number of linearly independent columns

Row / column $\rightarrow$ Same

	C_1	C_2	C_3	C_4	C_5
R_1	1	2	5	3	4
R_2	3	3	9	6	9
R_3	2	3	8	5	7
R_4	4	1	6	5	9

$\begin{matrix} \uparrow & \uparrow \\ R & C \end{matrix}$
 $C_3 = C_1 + 2C_2$
 $C_4 = C_1 + C_2$
 $C_5 = 2C_1 + C_2$

Rank = Col or Row
 same
 C_1, C_2
 independent one
 (2) Rank

What is a Rank of this matrix?

(2)

how many linear independent row or column are there?

ANS (2)

Rank 2

RANK = 2

$$\begin{bmatrix} C_1 & C_2 & C_3 & C_4 & C_5 \\ 1 & 2 & 5 & 3 & 4 \\ 3 & 3 & 9 & 6 & 9 \\ 2 & 3 & 8 & 5 & 7 \\ 4 & 1 & 6 & 5 & 9 \end{bmatrix}
 = \text{decompose} =
 \begin{bmatrix} C_1 & C_2 \\ 1 & 2 \\ 3 & 3 \\ 2 & 3 \\ 4 & 1 \end{bmatrix}
 \begin{bmatrix} 1 & 0 & 1 & 1 & 2 \\ 0 & 1 & 2 & 1 & 1 \end{bmatrix}$$

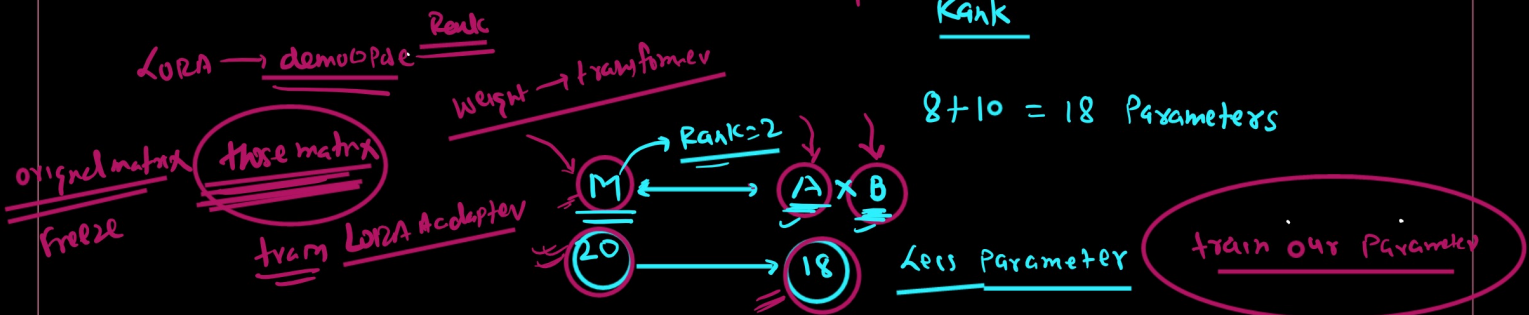
Actual

$4 \times 5 = 20$ Parameters

$4 \times 2 = 8$

Rank

$2 \times 5 = 10 = 18$

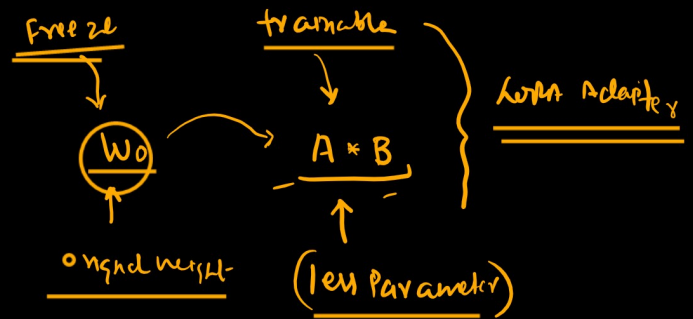
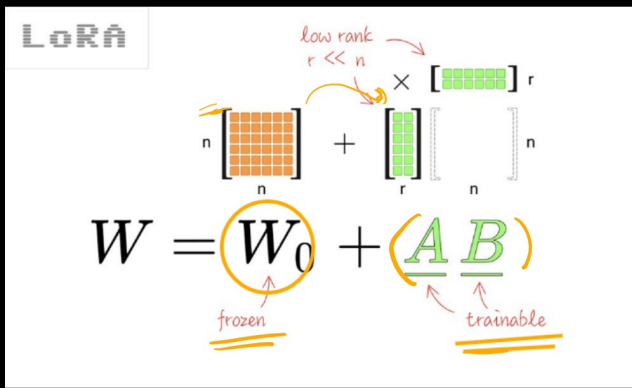


Low Rank $\rightarrow$ which is used in LORA

$$\begin{matrix} \boxed{W} \\ d \times k \end{matrix}
 =
 \begin{matrix} \boxed{A} \\ d \times r \end{matrix}
 \cdot
 \begin{matrix} \boxed{B} \\ r \times k \end{matrix}$$

$r < \min[d, k]$

Rank
 $W \rightarrow \text{LORA (Rank)} \rightarrow 4$
 $W \rightarrow (d \times 4) (4 \times k) \leftarrow \text{(trainable)}$
 4 | 6 | 5 | 10 | 12 | 16



LoRA trains small low-rank matrices instead of full weights

These weight changes are tracked in two separate, smaller matrices that get multiplied together to form a matrix the same size as the model's weight matrix.

LoRA says:

"It is possible that the weight update matrix is not full rank."

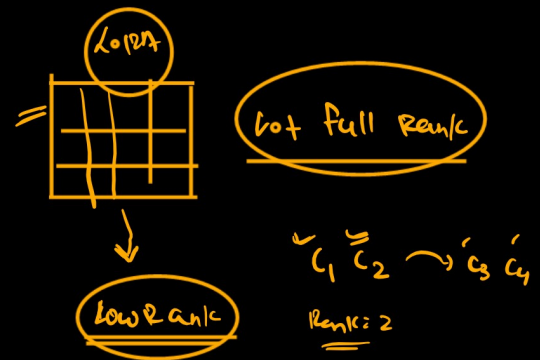
Its rank can be small.

LoRA: Learn two small matrices B and A.

And finally:

$$W_{\text{new}} = W_0 + BA$$

optimizer



Low-rank update = updating a large matrix by approximating it using smaller matrices.

LoRA

QLoRA(Quantized LoRA)

- This is LoRA 2.0
- Uses even less memory with "recoverable" quantization

Why we should use LoRA?

GPU memory requirements are drastically reduced

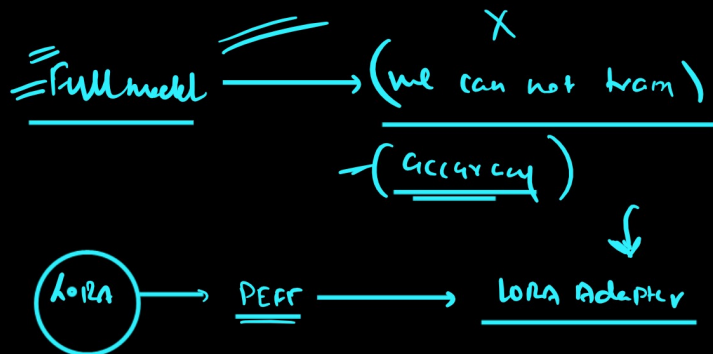
Training becomes much faster (even on a single GPU)

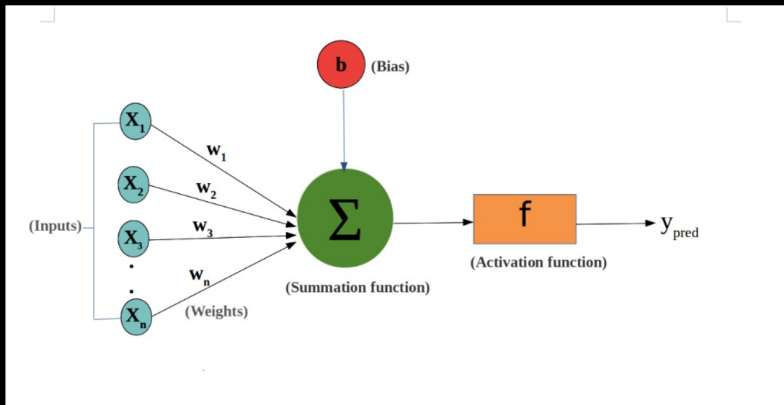
Fine-tuning becomes affordable (ideal for startups and individuals)

You can use different LoRA adapters for multiple tasks without duplicating the full model

Partial First

7B, 10B $\rightarrow$ LoRA





BP $\longrightarrow$ Back Propagation

Update weights using optimizers

GD / SGD: Simple update
 GD / SGD with Momentum: Smooth direction
 RMSProp: Adaptive learning
 Adam: Smart + fastest convergence
 AdamW(Weight Decay) + Learning Rate Scheduler + Warmup

GD (Gradient descent)

$$W_{\text{new}} = W_{\text{old}} - \eta \frac{\partial \mathcal{L}}{\partial W} \longrightarrow \mathcal{L} = (\gamma - \hat{\gamma})^2 \longrightarrow \hat{\gamma} = \sigma \left(\sum_{i=1}^n w_i x_i \right)$$

$$\downarrow$$

$$\mathcal{L} = \left[\gamma - \sigma \left(\sum_{i=1}^n w_i x_i \right) \right]^2$$

LoRA $\longrightarrow W = W_0 + \boxed{\Delta W}$

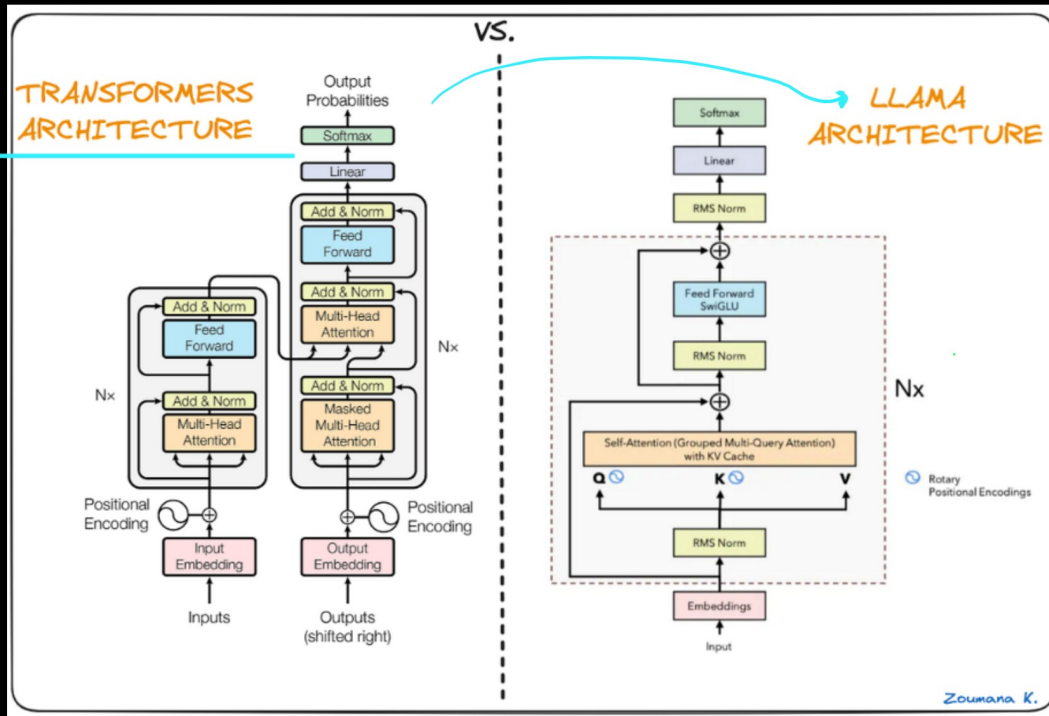
$W = W_0 + BA$

this Parameter (weight or adapter) will be updated

this is adapter
 or this is weight
 (Parameter)

How it will be updated using: AdamW
optimizer

Where is LoRA applied in LLMs(Where this adapter is going to be attached)?



What is a Transformer?

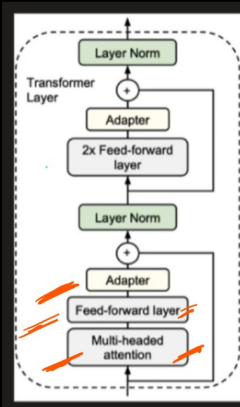
Attention + Neural Network

"A neural network architecture built around self-attention"
there:

- Loss is computed
- Gradient is calculated

In Transformer attention, matrices:

- W_q, W_k, W_v, W_o



Paper mostly applies LoRA on:

- W_q and W_v (query & value)

W_q
 W_v
 W_k
 W_o

→ LoRA

Best performance under same parameter budget:

- W_q alone → average
- W_v alone → average
- But $W_q + W_v$ → best overall

So practical rule:

q_proj + v_proj (most common industry default today too)

Model&Method	# Trainable Parameters	WikiSQL Acc. (%)	MNLI-m Acc. (%)	SAMSum R1/R2/RL
GPT-3 (FT)	175,255.8M	73.8	89.5	52.0/28.0/44.5
GPT-3 (BitFit)	14.2M	71.3	91.0	51.3/27.4/43.5
GPT-3 (PreEmbed)	3.2M	63.1	88.6	48.3/24.2/40.5
GPT-3 (PreLayer)	20.2M	70.1	89.5	50.8/27.3/43.5
GPT-3 (Adapter ^H)	7.1M	71.9	89.8	53.0/28.9/44.8
GPT-3 (Adapter ^H)	40.1M	73.2	91.5	53.2/29.0/45.1
GPT-3 (LoRA)	4.7M	73.4	91.7	53.8/29.8/45.9

175B → LoRA
4.7M.

Feature	LoRA	QLoRA
Model Precision	FP16 / FP32	4-bit (quantized)
Memory Usage	Medium	Very Low
Speed	Fast	Slightly slower
GPU Requirement	16-24GB	8-16GB enough
Library	PEFT	PEFT + bitsandbytes

Bits byte + PEFT

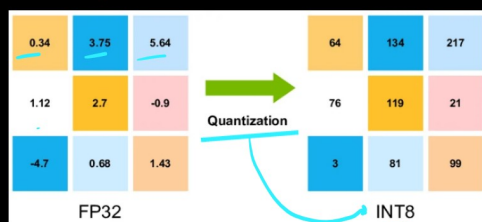
Parachute (QLoRA)

Type	Bits	Bytes
FP32	32 bit	4 byte
FP16	16 bit	2 byte
4-bit	4 bit	0.5 byte

3.1
 3.14 ← Approximate
 3.141
 3.14156 ← Actual

Decimal
 0.3
 0.4
 1.5
 2.9
 10.12 } memory

Full Precision
 ↓
 half Precision
 ↓
 INT



less memory

Weight

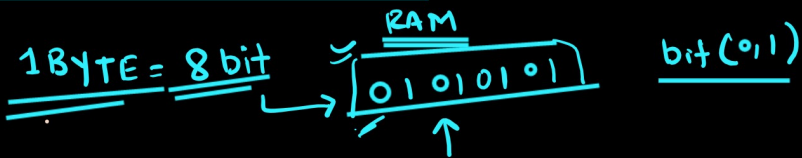
0.1234568
 0.12

FP

memory
 FP16 / FP32
 ↑ 2
 ↑ 4

Quantization is numerical compression of model weights; QLoRA uses 4-bit compressed base weights plus trainable LoRA adapters to reduce GPU memory while keeping fine-tuning effective.

$\text{FP32} = 32 \text{ bits} = 4 \text{ bytes}$ ← RAM memory
 $\text{FP16} = 16 \text{ bits} = 2 \text{ bytes}$
 $\text{INT8} = 8 \text{ bits} = 1 \text{ byte}$
 $\text{INT4} = 4 \text{ bits} = 0.5 \text{ byte}$



QLoRA combines two ideas: 1 bit

Quantization + LoRA

Normally FP16 means:

1 parameter = 16 bits = 2 bytes

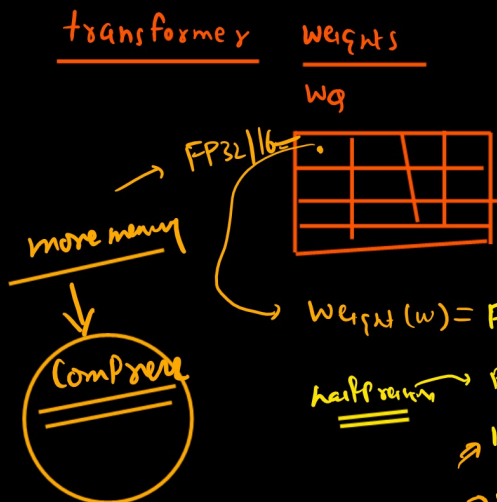
4-bit means:

1 parameter = 4 bits = 0.5 bytes

Example:

7B model in FP16 $\approx$ 14 GB

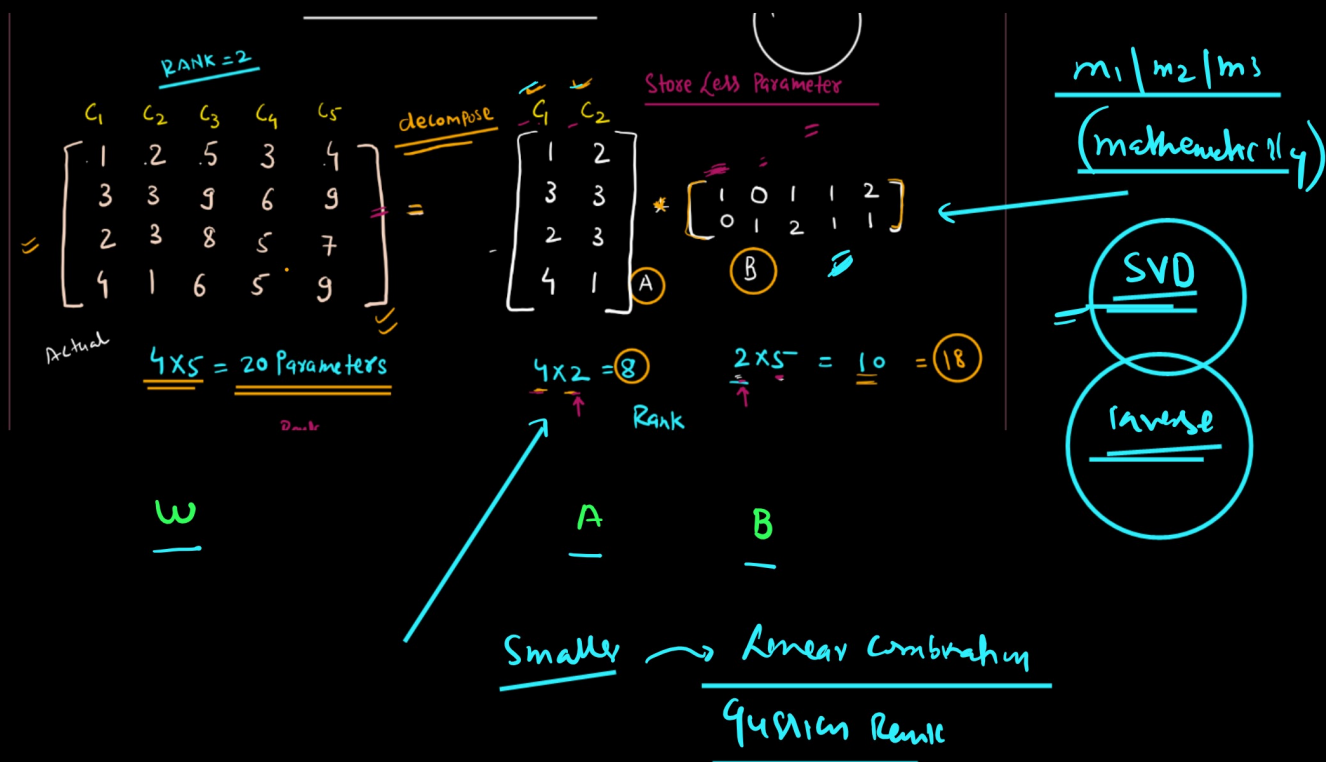
7B model in 4-bit $\approx$ around 3.5-5 GB with overhead



$\text{Weight}(w) = \text{FP32} \rightarrow 2.4328762$ (Full Precision)
 $\text{FP16} \rightarrow 2.432$ (Half Precision)
 $\text{INT8} \rightarrow -128 \text{ to } 127$
 $\text{INT4} \rightarrow -8 \text{ to } 7$

3.1415927
 3.141
3

Quantization



The flow is:

Install OpenAI SDK



Set API key



Create OpenAI client



Test base model



Load training data



Upload training file



Create fine-tuning job

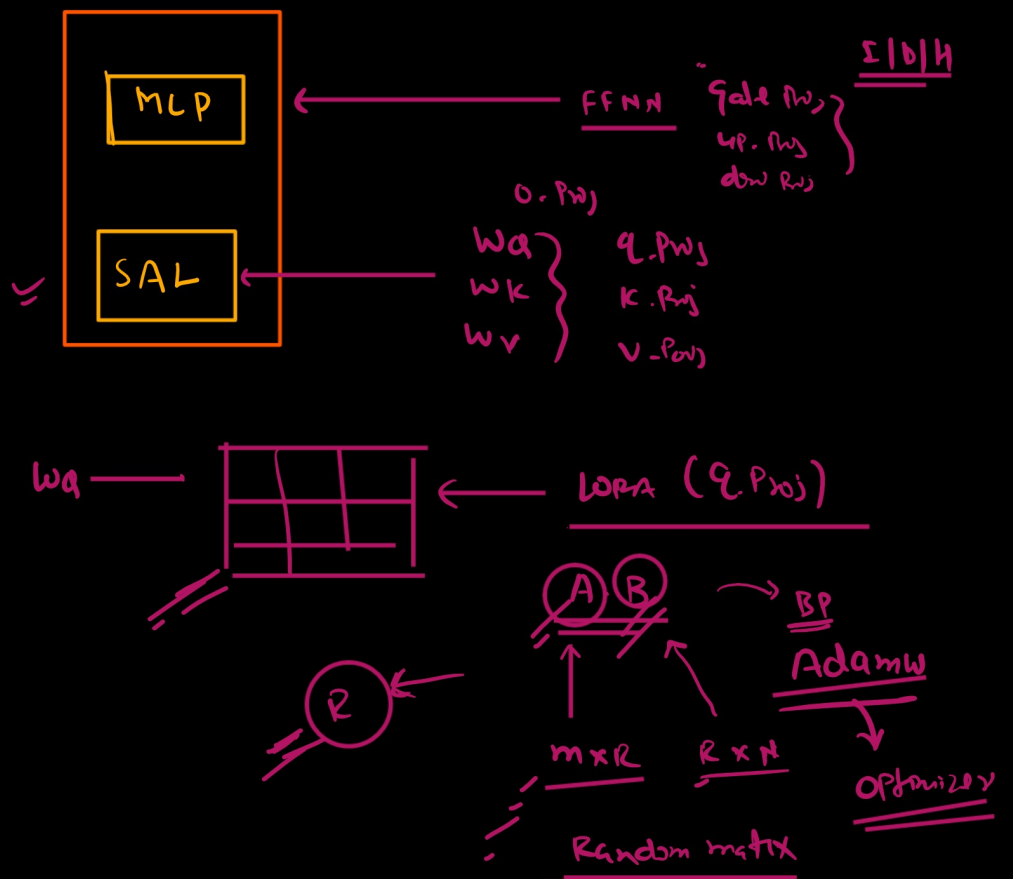


Check fine-tuned model



Use fine-tuned model for inference

`target_modules=["q_proj", "k_proj", "v_proj", "o_proj", "gate_proj", "up_proj", "down_proj"]`



Why Fine-Tuning Is Needed

Before fine-tuning, the base model is general-purpose.

It can answer many questions, but it may not follow our company-specific style, rules, tone, or domain behavior consistently. 

For example, suppose we are building a smartphone customer support bot.

We want the model to:

Answer only smartphone-related queries
Be friendly and concise
Avoid unrelated topics
Follow company-specific support style



We can try to do this using prompting, but if the behavior must be repeated again and again across many examples, fine-tuning can make the model learn that behavior more consistently.

There are different roles:

(system) → instruction / behavior setting
user → user question
assistant → model answer
Why is system message needed?

The system message controls the model's behavior.

Example:

You are a helpful assistant. / Medical assistant / Finance assistant
means the model should behave like a helpful assistant.

What happens if we remove system message?

The model can still answer, but we lose behavior control.



```
{ "messages": [ { "role": "system", "content": "You are a customer support assistant for a smartphone company. You are friendly, concise, and provide only factual answers related to smartphones." }, { "role": "user", "content": "Can I use the same apps on Android and iPhone?" }, { "role": "assistant", "content": "Not all apps are available on both platforms. Some apps are exclusive to either Android or iOS, depending on the app store." } ] }
```

General GPT model

↓

Training examples

↓

Fine-tuning job

↓

Custom GPT model

↓

Domain-specific customer support behavior

